

Chinchilla-Style Scaling Laws for a From-Scratch Arithmetic Transformer: Dense vs. Mixture-of-Experts

Nishant Relan

July 17, 2026

Abstract

I derived Chinchilla-style scaling laws for small (100k to 10M parameter) dense and MoE transformers trained from scratch on 3-digit addition. I found that the classic power law does not fit this setting. Rather than decaying smoothly, loss collapses at a sharp phase transition. Thus, I modeled the amount of data needed to trigger this transition at a given model size, and found that it scales as $D^* \propto N^{-0.34}$. Since this exponent is greater than -1 , the task is data-leveraged; the compute-optimal choice is a small model trained longer. Furthermore, I found that adding capacity at matched active parameters (MoE models) does not improve data efficiency. Rather, the scaling laws track active parameters.

1 Introduction

This report summarizes the results from deriving Chinchilla-style scaling laws for small dense and MoE transformer models. The goal for this project was to derive these scaling laws on a simple 3-digit addition task. These laws assumed a power law relationship between a model’s performance and its size, and derived coefficients for this power law describing how model size and data impact final performance. From there the authors derived a compute optimal frontier for how to scale both model size and data as compute budget scales. This power law held because, for natural language tasks such as pretraining, final loss decays smoothly as both model size and data increase.

In this project, I trained models ranging from 100k to 10M active parameters, and then derived the corresponding power law coefficients. I discovered the following. Firstly, the training loss curves do not exhibit a smooth decay. Rather, models undergo a phase transition where both loss and exact match accuracy go from poor to near-perfect over a small number of steps after a certain point in training. In particular, loss does not decay smoothly as data and model size increase. Instead, we see final loss clustering either at a high value, or very close to zero. Thus, while it is mathematically possible to derive the power law, it is not an accurate model of how performance scales in this setting, and I show this below. Secondly, I found that using a logistic surface to model the probability that a given model size and data horizon will lead to a phase transition was a much more accurate model of how performance scales. Modeling performance this way, I found that 3-digit addition is fundamentally a data-leveraged task. In particular, while increasing model size does help with data efficiency, it does not help enough to offset the compute cost. Thus, the compute optimal configuration lies at training a small model. Furthermore, adding capacity via MoE architectures while keeping active parameters constant does not help improve the data efficiency of the models.

2 Related Work

We first attempt to derive scaling laws using the same methodologies as in [1]. In particular, we follow their general experimental setup and the methodologies (where applicable) used to derive the power law coefficients.

The performance of transformers on algorithmic tasks has been studied in literature. The most relevant related work was by [2] which showed that when training transformers on addition, loss tends to drop to a plateau, where it stays for a while during training, before quickly and sharply dropping to near-zero. They connect this to low-rank matrix completion, showing how while the internal addition circuit is being formed, accuracy stays low, but once it is completed, accuracy rises sharply to near 100%. The paper also noted that formatting plays a big role in how quickly models learn as well. This is particularly relevant to this work, as almost all trained models exhibited this type of phase transition.

[3] explores the computational model of a transformer and analyzes the complexity of various algorithmic tasks using this model. In particular, they evaluate performance and behavior of transformers on a variety of algorithmic tasks, as well as the capacity of differently sized models.

3 Experimental Setup

3.1 Task and Data

Models were trained on simple two-operand, 3-digit addition. The universe of tokens included the digits 0 through 9, as well as +, =, a single white-space token, and an EOS and PAD token.

To keep the task simple, all operands were 0-padded to three digits, and the answer was 0-padded to four digits. This meant that each example had a sequence length of exactly 17 tokens (including the EOS token). All prompt tokens were loss-masked, and the model was only trained on the "answer" tokens (all tokens after the equals sign and space). This made the task completely deterministic, meaning that a loss of exactly 0 is achievable. In other words, the entropy floor for this task is 0.

3.2 Model Sizes and Data Horizons

For this task there are exactly 10^6 unique examples. Preliminary experiments showed that all models could fully learn this task with fewer examples than the full universe. Given that, the experiments used 8 horizons ranging from 8k to 400k examples. During a single training run, no example was ever seen twice by a model. Throughout, D denotes the number of *learned* tokens (the four answer digits plus the EOS token, i.e. 5 per example), not raw examples; these horizons thus span roughly 4×10^4 to 2×10^6 learned tokens.

The main sweep used 8 model sizes evenly log-spaced between 100k and 10M params. For MoE models, I set $k = 1$ (top-1 routing) to match active parameters with their corresponding dense counterparts, and ran the 8×8 grid at $E = 4$ and $E = 16$ to explore how sparsity impacts the scaling behavior.

For both the dense and MoE experiments, I ran the grid 5 times using 5 different seeds, in order to be able to model the stochasticity in a given model size phase transitioning at a particular data horizon.

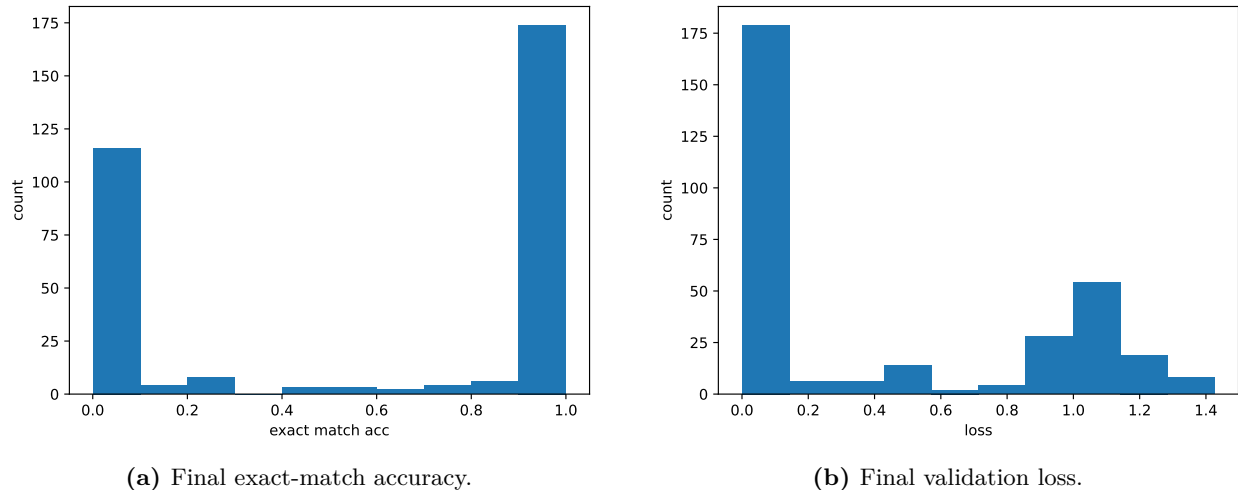


Figure 1: Distribution of loss and exact match accuracy across all dense runs. We find that the distributions are highly bimodal. This is one demonstration showing how sharply and quickly loss drops during the transition phase, given the small number of runs terminated mid-transition.

3.3 Other Hyperparameters

In line with [1], each run’s learning rate operated on a cosine schedule with a short linear warmup. The learning rate decayed by a factor of 10 over the course of training. I also used a fixed batch size of 16 for all experiments. The small batch size was to guarantee that even for the runs at the shortest horizons, there were still enough training steps for the model to learn. I also leveraged results from [4], [5], and [6] when designing the hyperparameter setup for the full sweep.

4 Dense Model Results

In this section we derive the Chinchilla laws using the final validation loss for each (dense model, data) pair in the grid. In particular, we use the experimental data to fit the following functional form as is done in Chinchilla:

$$L(N, D) = E + \frac{A}{N^\alpha} + \frac{B}{D^\beta} \quad (1)$$

4.1 Deriving Chinchilla Laws

4.1.1 IsoFLOPs approach

We start with the dense model experiments and show in multiple ways how the above functional form is ill-suited for accurately describing scaling behavior. Firstly, Figure 1 shows that final performance was highly bimodal, even when using a fairly densely spaced grid of data horizons and model sizes. We also plot the IsoFLOP slices in Figure 2. While we do see a bit of the nice U-shaped behavior for some of the lower compute slices, at higher computes, all models fully learn the task. In particular, the minimum loss at the high compute slices is ill-defined and thus it is impossible to derive the power law coefficients directly using the IsoFLOP approach.

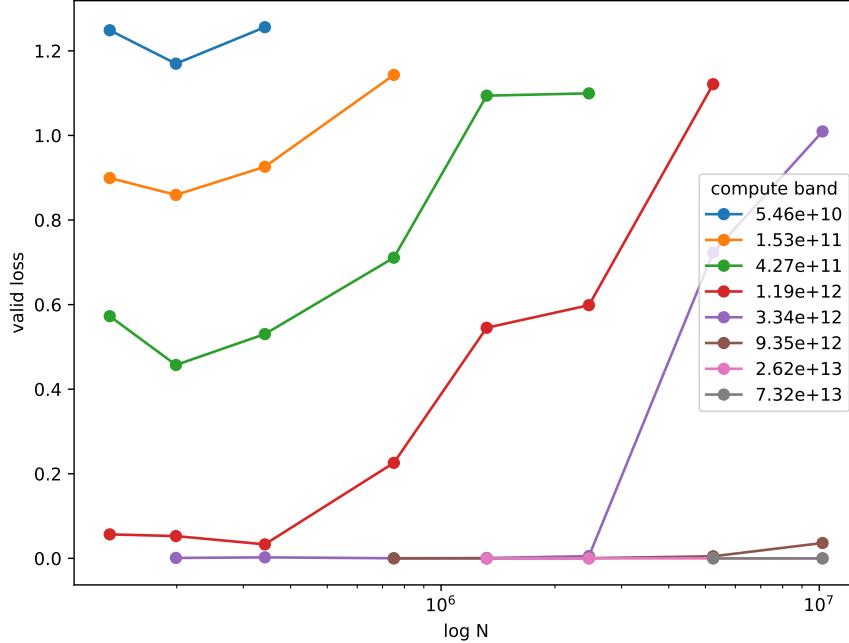


Figure 2: IsoFLOP slices: Loss vs. N within compute bands. At higher compute bands, all models solve the task and achieve near-zero loss, making the different configurations indistinguishable.

4.1.2 Fitting $L(N, D)$

While it would be difficult to even derive power law coefficients using the IsoFLOP approach, we can still directly fit the functional form and solve for A, B, α, β, E . We do so below using the same approach as Chinchilla – minimizing the Huber loss between the predicted and observed log-loss. When doing so, what we find is that even with a wide initialization grid, solutions for A, B, α, β all pin at or very near the top of their ranges. That is one signal the model cannot fit the data properly. Secondly, Figure 3 shows that the residuals of the fitted model are highly structured. At near-zero loss, the model underestimates the performance of the model, and then consistently overestimates the performance of the model at higher losses.

From both the IsoFLOPs and parametric loss fit, it is clear that the classic “Chinchilla” scaling laws do not accurately describe this setting.

We posit a different model to describe this setting instead, and describe it and its results in the next section.

4.2 More Accurate Scaling Laws

4.2.1 Fitting Optimal Data Horizon for Model Size

Due to the bimodal nature of results, and previous literature showing that for algorithmic tasks a phase transition during training is common, I argue that it is more useful to model the minimum amount of data needed to facilitate a phase transition at a given model size. I took two approaches to do so. Firstly, for each model size, I determined the data horizon at which the majority of the 5 runs had phase transitioned, where a phase transition is defined as exact match accuracy being above a threshold τ . If we call this horizon D^* , I am assuming the following relationship:

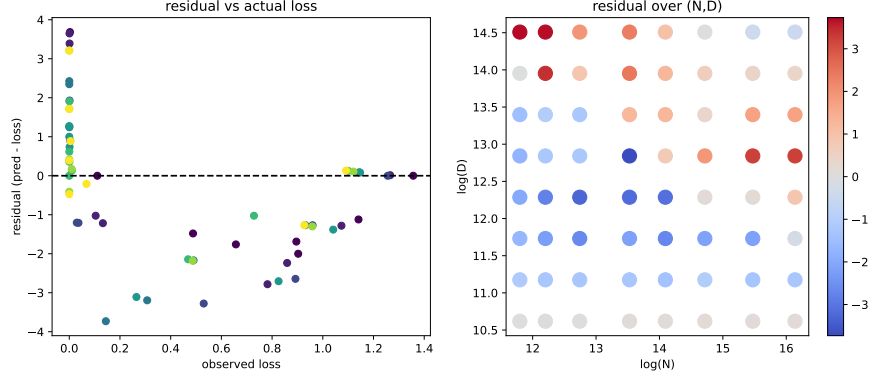


Figure 3: Residuals of the fitted $L(N, D)$ surface.

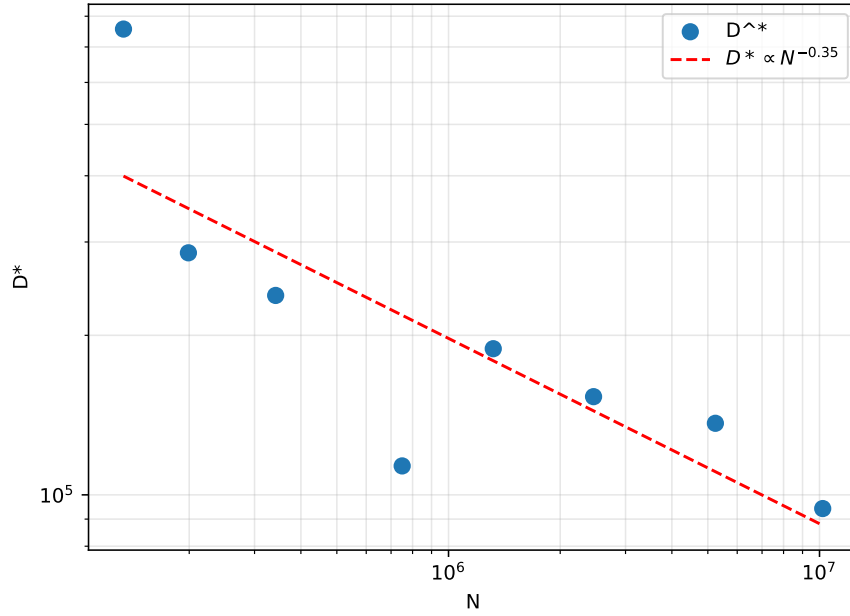


Figure 4: Fitted $\log D^*$ vs. $\log N$ for the dense models.

$$\log D^* \propto b \log N \quad (2)$$

When fitting to this model at $\tau = 0.5$, we find that $b = -0.35$. Figure 4 shows the full fit.

4.2.2 Fitting the Full Logistic Surface

The other approach I took was to fit the following parametric form.

$$P(A) = \sigma(b_0 + b_1 \log N + b_2 \log D) \quad (3)$$

where A is the event that the final exact match accuracy is greater than τ . Note that when setting $P(A) = 0.5$ and solving for $\log D$, we get

τ	D^* slope	D^* 95% CI	$-b_1/b_2$	$-b_1/b_2$ 95% CI
0.50	-0.345	$[-0.390, -0.154]$	-0.340	$[-0.407, -0.262]$
0.75	-0.363	$[-0.429, -0.295]$	-0.374	$[-0.420, -0.328]$
0.90	-0.375	$[-0.409, -0.289]$	-0.385	$[-0.417, -0.347]$
0.99	-0.477	$[-0.542, -0.346]$	-0.447	$[-0.519, -0.376]$

Table 1: Boundary slope b estimates for the dense models across solve thresholds τ , from both fitting approaches (direct D^* line fit and logistic $-b_1/b_2$), with bootstrap 95% CIs. Note that, while b drifts lower as τ increases, the two approaches’ predictions drift together.

$$\log D = \frac{b_0}{b_2} + \frac{-b_1}{b_2} \log N \quad (4)$$

When fitting to the above parametric form via logistic regression, we find that $b := \frac{-b_1}{b_2} = -0.34$. Thus, we find that these two distinct fitting methodologies yield very similar results predicting how the amount of data needed for the model to learn the task decreases as N increases.

5 Compute Optimal Allocation for Addition

If we set $C^* = 6ND^*$, then we find that

$$C^* \propto N^{1+b} = N^{0.65} \quad (5)$$

In particular, since $b > -1$, we see that the compute needed to train a larger model on this task rises with N . Thus, we find that this task is data-leveraged. In other words, increasing the amount of training data is much more compute efficient than increasing the model size. Thus, in order to use the least amount of compute to train a model to learn 3-digit addition, the derived law implies that the smaller the model, the more compute efficient it is.

This law obviously does not extend for extremely low N . In particular the model in question must have sufficient capacity to learn the task. In fact, if we directly look at the experimental data to determine the compute optimal model, we find that it is the second smallest model, at $\sim 200k$ parameters. While the smallest model still manages to learn the task, it takes disproportionately more data to do so, compared to what our derived law implies. This suggests that, at least in the given training setting, 100k params is near the capacity floor, and our derived laws start breaking down for $N \leq 10^5$.

6 MoE vs Dense Results

Given the results we found for dense models, the next question was whether or not the extra parameters at matched compute that MoE architectures provide could change the picture. To test this, I ran the same 8×8 grid at two additional sparsity levels. In particular, I set $k = 1$ to match active params at a given sparsity (not including router weights, which can be considered negligible), and ran the grid with 4 and 16 experts. To stabilize training I incorporated the auxiliary loss as in [7], and z-loss as in [8]. Given the low batch size of 16 examples per batch (288 tokens per batch),

N	C^* (FLOPs)
1.33×10^5	2.174×10^{12}
1.99×10^5	1.230×10^{12}
3.42×10^5	1.757×10^{12}
7.50×10^5	1.837×10^{12}
1.32×10^6	5.380×10^{12}
2.46×10^6	8.141×10^{12}
5.24×10^6	1.545×10^{13}
1.02×10^7	2.076×10^{13}

Table 2: Compute to solve $C^* = 6ND^*$ (FLOPs) for each dense model size N . While our law would predict the smallest model to be compute optimal, we find that the second smallest model at 200k params wins out. This could suggest a soft capacity floor around 100k params.

I found that going above 16 experts led to significant load balancing instabilities which would have confounded the analysis of how sparsity impacts the scaling laws.

Overall, the general trend was extremely similar to that of the trend with the dense models. In particular (and unsurprisingly), MoE runs also underwent a sharp phase transition, and as a result when fitting $L(N, D)$ I found similarly structured residuals. I omit the corresponding figures here, as they are virtually identical.

Based on the above I proceeded with the same alternative analysis which I did for the dense models. The results are in Table 4. From the results, we can draw some conclusions:

1. With MoE models, the task is still primarily data leveraged, with $-1 < b < 0$ for both sparsity levels.
2. Increased capacity at the same number of active parameters does not significantly improve data efficiency on this task.

In summary, we see that the scaling laws track the active parameters of the MoE models rather than the total parameters. To further validate some of the conclusions above, when examining expert health throughout training, we find that the extra capacity is in fact used. In other words, training does not collapse to a single expert, but rather experts are alive and load-balanced throughout training. Details are available in Table 3.

It is worth mentioning that, while in general we see the MoE scaling laws track active parameters, we see a couple of discrepancies between these results and the dense ones. In particular, $b_{\text{dense}} > b_{E=4} > b_{E=16}$, which would imply that on average, increased sparsity slightly increases sample efficiency compared to dense models at matched active parameters. Table 5 illustrates these results. It’s important to note that the MoE scaling law still predicts that small models are compute optimal for this task.

7 Limitations and Future Work

While the results above strongly indicate that scaling behavior on arithmetic tasks diverges from natural language tasks, it is important to address limitations of this work. In particular, it is possible that fixing $k = 1$ reduces the data efficiency ceiling for the MoE models in question. There

E	dead expert-slots (mean / max)	peak load / fair (mean / max)	routing entropy (mean / min)	runs with any dead
4	0.00 / 0	1.18 / 1.55	0.992 / 0.938	0%
16	0.11 / 2	1.82 / 3.85	0.976 / 0.897	71%

Table 3: Summary of expert health across all MoE runs.

τ	D^* slope	D^* 95% CI	$-b_1/b_2$	$-b_1/b_2$ 95% CI
0.50	-0.388	$[-0.447, -0.256]$	-0.371	$[-0.424, -0.313]$
0.75	-0.373	$[-0.447, -0.263]$	-0.372	$[-0.418, -0.329]$
0.90	-0.348	$[-0.491, -0.304]$	-0.377	$[-0.447, -0.330]$
0.99	-0.419	$[-0.527, -0.383]$	-0.482	$[-0.539, -0.423]$

(a) $E = 4$ experts

τ	D^* slope	D^* 95% CI	$-b_1/b_2$	$-b_1/b_2$ 95% CI
0.50	-0.434	$[-0.500, -0.276]$	-0.465	$[-0.518, -0.378]$
0.75	-0.468	$[-0.573, -0.361]$	-0.479	$[-0.595, -0.381]$
0.90	-0.431	$[-0.571, -0.348]$	-0.474	$[-0.582, -0.369]$
0.99	-0.457	$[-0.508, -0.369]$	-0.472	$[-0.521, -0.425]$

(b) $E = 16$ experts

Table 4: b estimates for MoE models with $E = 4$ and $E = 16$ experts. Note once again how the two estimates drift together as τ increases.

is literature showing (albeit on natural language tasks and at much larger scale) that $k > 1$ can yield performance gains [9, 10], and most modern models use $k > 1$ or dynamic k .

Another important limitation is that given the noisiness of the transition, results derived from 5 seeds are most likely noisy as well (in particular the bootstrapped confidence intervals for b). While the results tell a consistent story, running the sweep on more seeds would provide a more accurate view.

Interesting future work could explore or extend on interpretability efforts that aim to understand how transformer models actually form these simple arithmetic circuits. Furthermore, it may be interesting to explore models at even lower parameter counts (e.g. 10k–100k or even lower), where dense models hit their capacity floors, to see if MoE models (increased capacity at equal FLOPs) help performance in that regime.

8 Conclusion

I explored transformer scaling laws at small model scales on a simple arithmetic task and found that the behavior differs significantly from classic "Chinchilla-style" behavior. Therefore, I derived a separate law that more accurately captures scaling behavior for this particular setting, and showed that this is primarily a data-leveraged task. I also found that the increased capacity at equal FLOPs provided by MoE models does not help increase the models' data efficiency.

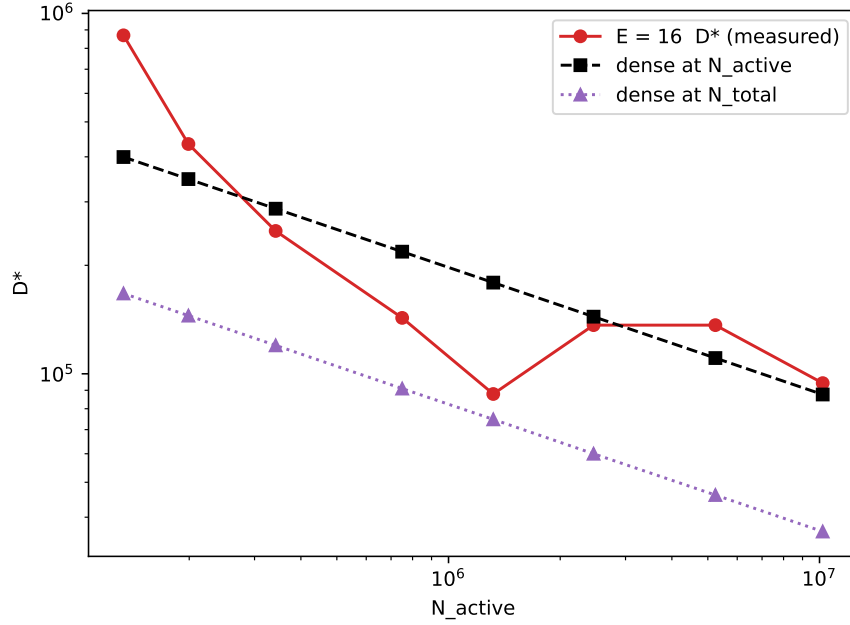


Figure 5: MoE D^* vs. the dense boundary evaluated at active and total parameter counts. Note that the purple line extrapolates the dense power law to well beyond the sizes of models that we trained. It mainly serves to illustrate the sample efficiency gap between a sparse and dense model at matched total parameters (rather than active).

N (active)	$E = 1$	$E = 4$	$E = 16$
1.33×10^5	2.174×10^{12}	1.646×10^{12}	2.497×10^{12}
1.99×10^5	1.230×10^{12}	1.866×10^{12}	1.866×10^{12}
3.42×10^5	1.757×10^{12}	2.429×10^{12}	1.840×10^{12}
7.50×10^5	1.837×10^{12}	2.660×10^{12}	2.316×10^{12}
1.32×10^6	5.380×10^{12}	2.880×10^{12}	2.506×10^{12}
2.46×10^6	8.141×10^{12}	5.367×10^{12}	7.251×10^{12}
5.24×10^6	1.545×10^{13}	1.143×10^{13}	1.545×10^{13}
1.02×10^7	2.076×10^{13}	3.149×10^{13}	2.076×10^{13}

Table 5: Compute to solve $C^* = 6ND^*$ (FLOPs) for each active model size N , at each expert count $E \in \{1, 4, 16\}$ ($E = 1$ is dense). Bold entries mark the compute-optimal (minimum- C^*) model size within each expert count.

References

- [1] Jordan Hoffmann et al. “Training Compute-Optimal Large Language Models”. In: *arXiv preprint arXiv:2203.15556* (2022).
- [2] Nayoung Lee et al. “Teaching Arithmetic to Small Transformers”. In: *arXiv preprint arXiv:2307.03381* (2023).
- [3] Gail Weiss, Yoav Goldberg, and Eran Yahav. “Thinking Like Transformers”. In: *arXiv preprint arXiv:2106.06981* (2021).
- [4] Shane Bergsma et al. “Power Lines: Scaling Laws for Weight Decay and Batch Size in LLM Pre-training”. In: *arXiv preprint arXiv:2505.13738* (2025).
- [5] Greg Yang et al. “Tensor Programs V: Tuning Large Neural Networks via Zero-Shot Hyperparameter Transfer”. In: *arXiv preprint arXiv:2203.03466* (2022).
- [6] Andrej Karpathy. *nanochat*. GitHub repository. 2025. URL: <https://github.com/karpathy/nanochat>.
- [7] William Fedus, Barret Zoph, and Noam Shazeer. “Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity”. In: *arXiv preprint arXiv:2101.03961* (2021).
- [8] Barret Zoph et al. “ST-MoE: Designing Stable and Transferable Sparse Expert Models”. In: *arXiv preprint arXiv:2202.08906* (2022).
- [9] Noam Shazeer et al. “Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer”. In: *arXiv preprint arXiv:1701.06538* (2017).
- [10] Quzhe Huang et al. “Harder Tasks Need More Experts: Dynamic Routing in MoE Models”. In: *arXiv preprint arXiv:2403.07652* (2024).